# Reading Less: Small-Model Context Selection for Long-Context MoE Inference on a Single 16 GB GPU

**Joseph Dominic Aruanno**

*Technical report draft, 2026-10-21. All numbers are from repository commit `972b928` and the artifacts it names; nothing here has been re-measured since.*

## Abstract

Long agent and chat sessions make a model re-read ever more context, and on consumer hardware a large mixture-of-experts (MoE) model pays for every token it reads. We study *SpecPrefill*, which uses a small student model that shares the target's tokenizer, attends over the whole conversation and chooses the few excerpts the large model reads on each turn. The target is Flash-Next, a hybrid MoE with 512 experts and top-10 routing, quantized to IQ3_S and served by the Strata engine with experts streamed from host RAM. The student is Qwen3.5-0.8B. Both run on one Tesla P100 (16 GB).

On a real 122-turn, ~253K-token Codex session, scored by exact-match quote probes, SpecPrefill is **1.93× faster end to end** than native Flash-Next with its prompt cache. It is faster at every depth bin (1.62–2.14×) and more accurate: **117/122 correct against native's 104/122**. Ordinary compaction at the same 48K target window scored 64/122. Flash-Next reads roughly 10–14% of the tokens native reads. On six held-out synthetic agent workloads it is 1.71–1.85× faster than native and 4.7–4.9× faster than compaction, but matches native's accuracy on only half of them.

The method does not help tool-heavy coding-agent loops. There each step's new input is a small tool result the model needs in full, and native's incremental prefill is already cheap. We report that negative result and the system design it led to. One Flash-Next engine resizes its VRAM at runtime: it serves agent loops natively and lends 8 GB to the student for the turns selection speeds up. On a live 11-task coding-agent evaluation this design passes 11/11 tasks (native 10/11) at a similar per-call cost overall.

Two findings may matter beyond this system. First, prefill cost on a streamed MoE is not linear in tokens read: any call of more than about 150 tokens pays roughly one full pass over the expert weights, which sets a floor on what selection can save. Second, the native baseline's long-context failures in our first runs came from int8 K/V quantization, not from the model.

## 1 Introduction

A coding agent's conversation grows with every file read, command output and reply. Serving engines use prompt caches so each turn only *prefills* the new tokens. With a large MoE on a small GPU, the per-turn cost is still dominated by prefill. The experts do not fit in VRAM, so each prefill call streams expert weights over PCIe. Long histories also strain the K/V budget: a 256K window and anything else co-resident compete for 16 GB.

Speculative prefill (Liu et al., 2025) proposes using a small model's attention to estimate which prompt tokens matter, and prefilling only those in the large model. We take this idea from single prompts to multi-turn sessions. We ask whether it holds up when:

- the conversation reaches 256K tokens;

- the student must attend over all of it on the same 16 GB card as the target;
- the system has to work as a drop-in OpenAI-compatible gateway for real clients.

Contributions:

1. **A multi-turn SpecPrefill design** in which the student holds the whole raw conversation in an append-only cache and the target sees only selected excerpts. The target never writes summaries, so nothing is lost and nothing waits on summary generation (§3).
2. **A cost model for prefill on a streamed MoE** that explains when selection can and cannot win (§3.4).
3. **Making a 256K student fit beside the target and run fast** on a Pascal GPU, including block-sparse student prefill that keeps selections intact (§4).
4. **Matched evaluations** against native cached inference and ordinary compaction: synthetic held-out workloads, a real 253K-token session, and a live coding-agent evaluation, with ablations (§5–§6).
5. **Negative results and the design they led to.** Selection does not help tool-heavy agent loops, and single-shot edits need grounding. These led to an elastic engine that moves VRAM between the target and the student at runtime (§6.4–§6.5).
6. **A production system** validated by a 1,077-turn soak with restarts and hard kills: zero errors and zero data loss (§7).

## 2 Background and related work

**Speculative prefill and token importance.** Liu et al. (2025) estimate token importance with a small model and pass only important tokens to the large model, on single prompts. We extend this to sessions: the student's state persists across turns, selection budgets track new input, and evidence that is delivered stays delivered.

**K/V cache compression and sparse attention.** H2O (Zhang et al., 2023) and SnapKV (Li et al., 2024) evict or compress the target's own K/V. Quest (Tang et al., 2024) bounds per-block keys to choose which K/V pages to attend to. MInference (Jiang et al., 2024) uses sparse prefill patterns. These methods reduce the target's attention cost but still require it to read every token once. On a streamed MoE that first read is the dominant cost (§3.4). We borrow Quest's per-block key bounds for the *student's* sparse prefill (§4).

**Prompt compression.** LLMLingua and LongLLMLingua (Jiang et al., 2023; 2024) drop tokens with a small language model before the large model sees the prompt. SpecPrefill differs in three ways:

- selection is per turn over a persistent, lossless archive;
- excerpts are line-aligned and carry their source labels;
- anything not shown stays available for later turns.

**Compaction.** Agent clients such as Codex summarize their history when the window fills. We use this as a baseline at the same target window, with its summaries written by the target model.

**Retrieval.** Retrieval-augmented generation fetches chunks by embedding similarity. We did not run a retrieval baseline (§8). The student's attention scores condition on the whole conversation and the current request, which an independent embedding does not.

## 3 Method

### 3.1 Roles

- **Target:** Flash-Next Swift IQ3_S on Strata. It is hybrid: sparse attention and recurrent layers keep its long-context cost nearly flat in depth. The weights are unchanged throughout this work.

- **Student:** Qwen3.5-0.8B. It shares the target's 248K-token vocabulary and has a native 262K context, with 6 of 24 layers full attention and 2 K/V heads.

The student keeps the raw conversation in an append-only K/V cache. Each turn it reads the new input, then runs one scoring trace for the current request and scores every chunk not yet delivered to the target.

### 3.2 Selection (`SpecSession`)

- **Chunks:** line-aligned, about 16 tokens. Long lines are split softly so numbers and identifiers stay whole.
- **Budget:** scales with *new* input, `0.06 × new + 160` rendered tokens per turn, framing included, with a confidence stop at 0.3 × the third-best chunk. An earlier version set the budget over the whole history and delivered any chunk touched by a selected position. It stopped pruning: the target read 135% of native's tokens (§6.6).
- **Framing:**
  - excerpts carry `[path vN]` headers, adding `superseded by vM` once a newer version exists, and never a "current" claim that can go stale;
  - a scope anchor gives each excerpt's enclosing definition line;
  - a one-line manifest lists the latest version of each file the request names.
- **Ranking priors:**
  - text the request quotes verbatim ranks first;
  - path and version bindings tie excerpts to the files and revisions the request names.
- **One-message turns:** each turn's excerpts and question form a single user message. Split into two messages, Flash-Next often replied with nothing when the excerpts ended on a shell command (65 of 79 filler turns in one replay).

### 3.3 No summaries; nothing lost

The target never writes summaries. If its window fills, it is rebuilt from a student-selected subset of the archive. With a 48K target window this never happened at 256K, because delivered evidence is about 10–14% of the conversation. The archive is lossless (SQLite per thread). Anything the target has not seen can be selected on a later turn.

### 3.4 Cost model: why selection is not free on a streamed MoE

Flash-Next routes each token to 10 of 512 experts per layer. Any prefill call of more than about 150 tokens therefore touches almost every expert and pays roughly one full pass over expert weights. Measured on the P100:

**Table 1.** Median Flash-Next prefill time on the P100 by fresh tokens read in one call.

| Fresh tokens read | Median prefill |
|---:|---:|
| ≤ 100 | ~1.9 s |
| ~230 | ~3.9 s |
| 400–800 | ~5.9 s |
| ~1,500 | ~8.1 s |
| ~2,400 | ~11 s |

So savings come from turns with large new input. A 2× win needs native turns of ≥8 s (≥1.5–2K new tokens) while the target reads ≤ ~200 tokens. A pure recall turn, where native reads ~50 tokens, cannot be won: SpecPrefill adds a student pass and roughly doubles its cost. At 256K the student, not the target, sets SpecPrefill's speed: native's cost per 2.3K-token turn barely grows with depth (11.1 s at 35K, 13.3 s at 256K).

## 4 Making a 256K student fit and run fast

The student runs on the same P100 as the target. Flash-Next is started with an 8 GB VRAM reserve for the student's 256K cache (peak about 6.3 GB). Without it, Strata's automatic expert cache absorbs any VRAM a smaller target window frees.

Kernel changes in `chunked_student.py`, in order of impact. Selections are identical to the original code (overlap 1.0 on both dev replays):

1. Lookahead decode attention used a kernel that parallelizes over query rows. With 4 rows against 256K keys the GPU sat idle for 5.8 s of a 14.9 s turn. It is now plain matrix multiplies (exact).
2. Prefill over the cached past runs as a tiled online softmax (the FlashAttention recurrence; Dao et al., 2022) on FP16 GEMMs. It is merged exactly with the causal current block via log-sum-exp, with error $\leq$ 2e-5 vs FP32.
3. Grouped-query attention without copying K/V to every head. The copy cost ~1 GiB per call at 128K.
4. Lookahead snapshots the recurrent state and trims K/V instead of deep-copying the cache.
5. K/V grow in a preallocated buffer.

Student time per 2.3K-token turn fell from 14.8 s to 7.5 s at 256K (2.4 s → 1.6 s at 32K).

**Block-sparse student prefill.** Each prefill block attends exactly to the 16,384 history keys with the best per-block key bounds (after Quest) plus the last 4K keys. Scoring rows and lookahead stay dense. Choosing the blocks once per 2,048-token prefill block, rather than every 256 query rows, turns attention into larger matrix multiplies: 105 ms vs 260 ms per layer call at 256K. Student-only replay over the 122-turn session ("gold" = the answer line delivered intact):

**Table 2.** Student-only replay of the 122-turn session under sparse-prefill settings. Gold: answer line delivered intact; Jaccard: selection overlap with dense prefill.

| Setting | Gold | Student s | Last 30 turns (s) | Jaccard vs dense |
|---|---|---|---|---|
| Dense | 119 | 375 | 100.5 | 1.00 |
| 16K keys, per 256 rows | 119 | 277 | 53.1 | 0.72 |
| 16K keys, per 1,024 rows | 120 | 183 | 36.1 | 0.70 |
| **16K keys, per 8,192 rows (chosen)** | 119 | 166 | 33.5 | 0.73 |
| 8K keys, per 1,024 rows | 120 | 154 | 31.8 | 0.62 |
| 4K keys, per 1,024 rows | 120 | 138 | 29.3 | 0.60 |

Sparse prefill changes about a quarter of the selected chunks but not which answer lines arrive. **Windowed** student prefill was tested and rejected: an 8K window over a 27K history lost 3/30 gold fields and changed ~40% of selections. The student must attend over everything.

## 5 Experimental setup

**Hardware.** One Tesla P100 16 GB (Pascal, no tensor cores). 62 GB of host RAM, of which Flash-Next's streamed experts pin about 47 GB.

**Arms:**

- **Native:** cached Flash-Next with a 262,144-token window and the whole GPU.
- **Compaction:** the same model at a 48K window, the window Flash-Next has under SpecPrefill. Flash-Next writes the summaries.
- **SpecPrefill:** a 48K Flash-Next window and a 256K student.

Every arm keeps its own replies (no teacher forcing). End-to-end time is everything after model load: student, selection, archive, any compaction, target prefill, decode and persistence.

**K/V precision.** Flash-Next's baseline configuration used int8 K/V. With it, native returned empty replies past about 50K tokens, and a cascade followed (§6.2). All final comparisons use FP16 K/V for both arms. That is *less* quantization than the baseline, fits a 262K native window, and costs native about 3%.

**Workloads:**

1. **Synthetic agent sessions** (`agent_cases.py`): 30 scored fields per session, mixing large tool and file inputs, revisions, late recall and multi-hop. Seed 777 is the dev set; seeds 9001–9006 were held out and never tuned on.
2. **A real session** (`log_cases.py`): a 2026-10-08 Codex session of the author's, 240,765 source tokens (~253K of conversation), with 122 probes:
   - 101 on the newest material;
   - 9 in-session recalls at least 20 turns back;
   - 12 late probes, 4 of them from the first 50K tokens.

   Each probe asks for a complete output line of a named command, unambiguous in the whole transcript, and is scored by exact match. Secret-looking lines are excluded, and the case stays local.
3. **A live coding-agent evaluation** (`live_agent_eval.py`): Codex CLI 0.162.1 fixes 11 planted one-line bugs in a copy of this repository, one task after another in one thread. It first reads about 125K tokens of unrelated code, so the tasks run at 128K+. A task passes when its test file passes and no test was edited. Thinking is on, as in native's default.

# 6 Results

## 6.1 Synthetic held-out agent sessions

8K target window, 32K student window. No run filled either window.

**Table 3.** Held-out synthetic agent sessions (30 scored fields each; seeds never tuned on). End-to-end wall time.

| Seed | Native | Compaction 8K | SpecPrefill | vs native | vs compaction |
|---|---|---|---|---|---|
| 9001 | 30/30, 142.5 s | 17/30, 377 s | **30/30, 78.1 s** | 1.82× | 4.8× |
| 9002 | 30/30, 143.4 s | 21/30, 383 s | 27/30, 77.7 s | 1.85× | 4.9× |
| 9003 | 30/30, 146.4 s | 20/30, 383 s | 28/30, 81.2 s | 1.80× | 4.7× |
| 9004 | 30/30, 143.9 s | 24/30, 388 s | **30/30, 80.1 s** | 1.80× | 4.8× |
| 9005 | 30/30, 143.1 s | 20/30, 389 s | 27/30, 83.5 s | 1.71× | 4.7× |
| 9006 | 30/30, 142.6 s | 19/30, 381 s | 29/30, 79.9 s | 1.78× | 4.8× |

- The target read 10–12% of native's tokens.
- SpecPrefill beats compaction on every seed, on both accuracy and speed.
- It is within one field of native on three of six seeds.
- Every miss had its gold line *delivered*. The target failed to use an excerpt shown turns earlier when asked for the "current" revision.
- A free student would reach only about 2.25× here, set by the target's per-call floor (§3.4).

## 6.2 The real 256K session

Final run, on the commit this report pins. The native and compaction runs are shared with the Phase 4 measurements.

**Table 4.** The real ~253K-token session by depth bin: probes correct and end-to-end wall time. Native and SpecPrefill use FP16 K/V; the compaction arm is an earlier run on the same case.

| Depth bin | Native 262K, FP16 | Compaction 48K | SpecPrefill | Speed-up vs native |
|---|---|---|---|---|
| 0–64K | 25/29, 466.6 s | 25/29, 517 s | **28/29, 230.8 s** | **2.02×** |
| 64–128K | 23/27, 462.0 s | 15/27, 476 s | **26/27, 215.6 s** | **2.14×** |
| 128–192K | 28/33, 531.8 s | 18/33, 581 s | **33/33, 268.6 s** | **1.98×** |
| 192K+ | 28/33, 406.6 s | 6/33, 389 s | **30/33, 250.5 s** | **1.62×** |
| **Total** | 104/122, 1,867 s | 64/122, 1,962 s | **117/122, 965.5 s** | **1.93×** |

- The compaction arm comes from an earlier run on the same case (§8).
- The same configuration gave 1.94× and 117/122 at Phase 4. This reproducibility check differs by at most 0.01× per bin.
- SpecPrefill had zero compactions, rebuilds or out-of-memory errors, and the student attended over the full 253K history.

Recall of older facts, from the same run:

**Table 5.** Probe classes on the real session (same runs as Table 4).

| Probe class | Native | Compaction | SpecPrefill |
|---|---|---|---|
| Newest material | 85/98 | — | **96/98** |
| In-session recall ( ≥ 20 turns back) | 6/9 | 0/9 | **8/9** |
| Late block | 10/12 | 0/12 | 10/12 |
| Facts from the first 50K tokens | 4/9 | 0/9 | **8/9** |

**The int8 K/V finding.** With int8 K/V, native was perfect for the first 15 turns, then truncated its replies. From about 145K tokens, every reply was a single end-of-turn token: 39/122 overall. Diagnosis (`cold_check.py`):

- At ~59K, the identical prompt answered correctly from a cold prefill but was empty on the cached run.
- At ~147K, a cold prefill with native's own mostly empty history stayed empty, but answered correctly once that history held correct replies.

So the collapse was a cascade: the model imitated its own empty replies, after a cached-path divergence near 50K. FP16 K/V removed it. Over turns 1–54, int8 scored 36/54 with 12 empty replies, and FP16 scored 46/54 with none. This matters for any long-context comparison that runs the baseline with quantized K/V.

### 6.3 Ablation history on the real session

**Table 6.** Ablation history on the real session. The first two speed-ups are against the int8-K/V native baseline.

| Version | Change | Correct | Speed-up |
|---|---|---|---|
| v3 | fixed probes | 100/122 | — |
| v4 | + quoted-span binding | 112/122 | 1.54× (vs int8 native) |
| v5 | + block-sparse student prefill | 115/122 | 1.68× (vs int8 native) |
| final | FP16 K/V for both arms | 116/122 | 1.76× |
| final | sparse blocks per 8,192 rows | 115/122 | 1.92× |
| Phase 4 | grounded edits, fresh tool results, one-message turns, thinking parity | 117/122 | 1.94× |
| Phase 6 | production hardening | 117/122 | 1.93× |

### 6.4 Negative result: tool-heavy agent loops

In a live agent loop, plain SpecPrefill was **slower** than native: 26.2 vs 18.4 s per model call below 64K, and 25.5 vs 18.2 s at 64–128K. Each step's new input is a small tool result the model needs in full, so selection has little to cut. Meanwhile SpecPrefill adds:

- a student pass (1–4 s);
- 0.7–0.9K excerpt tokens per call;
- occasional window rebuilds;
- more generated tokens per call (about 300 vs 200).

Flash-Next's incremental prefill barely slows with depth on this card (about 125 tokens/s at 222K), so native's cached path is hard to beat there.

**Replayed edits are not an agent.** The first coding evaluation asked the target for the next edit of a recorded session, with no tools. Blind side-by-side grades by a separate model that never saw the arm labels:

**Table 7.** Replayed-edit evaluation: blind side-by-side grades (correct + half of partial) by a separate model.

| Variant | Acceptable edits | Native |
|---|---:|---:|
| Sparse selection only | 6.7% | 56.7–60% |
| Larger budget (`new_rate` 0.25) | 6.7% | 56.7% |
| + grounding v1 (12K fetch) and one-message turns | 30% | 60% |

- A larger budget does not fix edits. Grounding doubles edit quality, but its 12K-token fetches cost about 70 s each.
- That led to *region grounding*: draft ~12 lines, find the hunk in the archive, and show only that region (±40 lines) before writing the change. Evidence shown this way is not kept.
- Because no real agent edits blind, the evaluation moved to live agent loops (§5).

### 6.5 The elastic engine: native for agents, the student when it helps

A 256K native window and a resident 256K student need about 21 GB of VRAM, against the card's 15.9. We added a runtime command, `VRAM <reserve_mib> [drop]`, to Strata's growable K/V mode (`--kv-grow`). It frees, in order:

1. spare chunks;
2. K/V cells past the live session;
3. expert-cache slots;
4. the lower part of the prompt-chunk buffer.

It later maps the memory back and refills the experts. Measured: 8.2 GB freed in 0.43 s and given back in 0.79 s, with the live session intact.

The gateway runs one Flash-Next with a 256K growable window:

- **Agent requests (tools declared)** are answered natively from the cached conversation, up to 200K tokens.
- **Chat turns** park the native conversation, free 8 GB, load the student (0.4–0.6 s) and run lean.
- **Turns whose new input is only fresh tool results** skip the student.

Deep live agent evaluation, 11 tasks each. Seconds per output token, by depth calibrated on native's token counts:

**Table 8.** Deep live coding-agent evaluation (11 tasks per arm): seconds per output token by conversation depth, tasks passed and mean seconds per model call.

| Depth | Native | Elastic (1,280-token chunk) | Elastic final (auto chunk) |
|---|---|---|---|
| 64–128K | 0.185 | 0.207 | 0.101 |
| 128–192K | 0.079 | 0.093 | 0.090 |
| 192K+ | 0.111 | 0.104 | 0.091 |
| **Tasks passed** | 10/11 | 11/11 | 11/11 |
| Mean s per call | 28.3 | 26.3 | 26.7 |

- Elastic matches or exceeds native's quality: it passed the task native timed out on.
- It is faster per output token above 64K except at 128–192K. There the growing K/V takes the VRAM a large prompt chunk would use, and native leads by about 14%. That is the 16 GB card's limit.
- The final engine found and fixed an out-of-memory failure. At ~187K the K/V could not grow beside an 8,192-token prompt buffer. The engine now shrinks that buffer before growing the K/V, tested at 195K.
- Codex does not pin sampling, so the arms took different paths (197–270 calls). Per-call cost and pass rate are the comparisons, not total time.

**Stacking with expert reduction.** In a side test, a research build used 2–3 routed experts per token during prefill. It sped up lean turns 14–24% but native only 0–11%. Lean's turns are mostly the target's fixed per-call prefill, which fewer experts cut, so the two techniques stack.

### 6.6 What broke the first design

The first integration added target-written summaries, provenance JSON, indexed retrieval and exact lookup, then stopped measuring speed. A per-turn token ledger found that:

- the selector had stopped pruning: the target read 135% of native's tokens at 32K, and 170% at 8K;
- compaction put the target on the critical path, with a 56 s summary call at one turn.

The lean design (§3) came from removing those pieces. On the dev seed it took the system from 1.7× *slower* than native to 1.81× faster, at 11% of native's tokens.

## 7 Production system

The design ships as an OpenAI-compatible gateway (`/v1/chat/completions` and `/v1/responses`). Codex, OpenCode and the OpenAI SDK run through it with no headers and no changes.

- **Thread identity** comes from the history itself, using chained prefix fingerprints. It handles forks, edited histories and retries.
- **Native fallback.** Anything the lean path can't serve (images, forced tool choices, a SpecPrefill error, a conversation under 1,024 tokens) goes to native with the same response shape.
- **Thread switches and resumes.** Per-thread student and Flash-Next state is saved to RAM-backed and disk caches. Switching back after a turn costs 4.5–5.0 s, against 286–297 s to re-read; resumed answers matched token for token in 5/5 trials.
- **Storage.**
  - Lossless archives are user data, with list, export, delete and quota controls.
  - A delete leaves no recoverable text.
  - A full disk or a failed write fails closed to native.
- **Operations.**

    - Queue limits with a clear busy error.
    - Per-request timeouts and cancellation on client disconnect.
    - Metrics without conversation text, and format fixtures for every release.
- **Defaults.** Lean is the default engine and `native` a one-setting opt-out. A one-command install auto-configures from a table of measured cards, and the student download needs consent and is checksum-verified.

**Soak.** We replayed 12 real Codex sessions on the final install: six as plain chat, served lean (494 turns), and six as agent loops with tools, served natively (583 turns). The run covered 1,077 turns and 81 thread switches, with 10 clean restarts and 2 `kill -9`s mid-generation. It had **zero errors**, and after it every thread continued its own stored history, so there was **zero data loss**. VRAM and resident memory stayed flat, and disk stayed within its caps (11.8 GB).

The soak also exposed a startup cost that grew with the square of stored messages (an FTS5 join on an unindexed column). Opening the store took over 20 minutes at 40K events; after the fix it takes 0.2 s.

## 8 Limitations and threats to validity

- **One real session.** The 256K result is one 122-turn session from one author's Codex use. The six synthetic held-out sessions come from one generator. Results need more sessions, from different agents and users, with confidence intervals. With 122 probes, the 13-field gap to native (117 vs 104) is large, but its variance across sessions is unknown.
- **Exact-recall probes.** The 256K benchmark scores exact quotes. It does not measure reasoning over the whole history. Whole-history requests are routed to native when the conversation fits, which is a design choice rather than a result.
- **The compaction baseline** at 256K came from an earlier SpecPrefill version's run on the same case. It was not re-run with FP16 K/V. It is a conservative stand-in for client-side compaction, not a tuned one.
- **No K/V-compression or retrieval baseline.** SnapKV-, H2O- or Quest-style target-side methods and embedding retrieval were not compared. On a streamed MoE, target-side methods still pay the first full read (§3.4), but that argument is not a measurement.
- **One model pair on one machine.** Flash-Next IQ3_S and Qwen3.5-0.8B on a P100 with 62 GB of RAM. The cost model (§3.4) depends on experts streaming over PCIe. On a GPU that holds all experts, prefill is cheaper and the margin smaller.
- **The live agent evaluation** is 11 tasks, one run per arm, with unpinned sampling. Small per-bin differences are within run-to-run variation.
- **The soak** ran 1,077 turns, short of the planned 1,500, because the replayed sessions held only that many answer points at the harness's settings. Its replies were capped at 48 tokens, so it shows stability, not speed.
- **The probe form matters.** An earlier "complete the rest of the line" probe made native collapse into ever-shorter replies, and we replaced it. Probe design can favour or punish either arm.

## 9 Conclusion

A small same-tokenizer student that attends over a whole long conversation can choose what a large MoE reads. On a real 253K-token session this makes the large model about twice as fast end to end on one 16 GB card, and more accurate than native cached inference and compaction. The gains come from turns with large new input, and they are bounded by the target's per-call prefill floor. In tool-heavy agent loops native's incremental prefill already wins, so a practical system routes those natively and moves VRAM to the student only when selection pays. The most transferable lessons are:

- measure the per-turn token ledger and end-to-end time on every change;
- model prefill cost on streamed MoEs as a per-call floor, not as linear in tokens;

- check K/V quantization before attributing long-context failures to the model.

## Reproducibility

- **Code:** this repository at `972b928`.
  - Method: `memory_gateway/specprefill/`.
  - Harnesses: `experiments/strata_specprefill/`, including `three_way_compaction.py`, `log_cases.py`, `depth_report.py`, `lean_replay.py`, `live_agent_eval.py` and `soak.py`.
  - The Strata patch: `integrations/strata/elastic-vram.patch`.
- **Per-depth reports:** `artifacts/strata-specprefill/`, in particular `log256k-phase6.json`, `log256k-final.json` and `log256k-20261017/depth_report_fp16_r8192.json`.
- **Not included:** the raw 256K case and the live-eval runs are kept outside the repository (`~/.cache`), because they contain the author's session.
- **Configuration:** a 48K Flash-Next window, 262,144-token student window, 8 GB reserve, FP16 target K/V, 16,384 sparse keys and 8,192 sparse rows.
- **Records:** `docs/specprefill-goal.md` (research log) and `docs/specprefill-production-plan.md` (production gates and measurements).

## AI contribution statement

This work was carried out with AI coding agents, under the author's direction:

- Claude Code (Anthropic's Claude models), OpenAI's Codex and Freebuff;
- the production phases (§7) and this report were done with Claude Code running Claude Opus 5.5.

**What the agents did:**

- wrote most of the code: the gateway, the selection method, the student kernels, the Strata engine patch, the test suites and the evaluation harnesses;
- ran the experiments and the soak;
- diagnosed failures;
- drafted the project's records and this report.

**What the author did:**

- set the goals and the pass/fail gates;
- made the design and method decisions the report describes, among them:
  - replacing replayed edits with a live agent evaluation;
  - routing agent loops natively and chat turns to SpecPrefill;
  - pursuing a runtime VRAM-sharing design instead of restarting between modes;
- decided when a gate passed with a documented limit, including accepting the shortened soak (§8);
- reviewed the results.

**AI in the evaluation itself:**

- Three AI systems appear as subjects of the evaluation, not as contributors. The models under study (Flash-Next and Qwen3.5-0.8B) are the system being measured. Codex CLI is the coding agent driven in the live evaluation (§5).
- The blind grades of replayed edits (§6.4) came from a separate Claude Sonnet agent, which never saw which arm produced which edit.

Every reported number comes from the scripts and artifacts listed under Reproducibility, not from model-written estimates. The author takes responsibility for the content.

## References

*To verify before submission.*


Dao, T., Fu, D. Y., Ermon, S., Rudra, A., Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. NeurIPS.

Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y., Qiu, L. (2023). LLMLingua: Compressing prompts for accelerated inference of large language models. EMNLP.

Jiang, H., et al. (2024). LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. ACL.

Jiang, H., et al. (2024). MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. NeurIPS.

Li, Y., et al. (2024). SnapKV: LLM knows what you are looking for before generation. NeurIPS.

Liu, J., et al. (2025). Speculative prefill: Turbocharging TTFT with lightweight and training-free token importance estimation. ICML.

Tang, J., et al. (2024). Quest: Query-aware sparsity for efficient long-context LLM inference. ICML.

Zhang, Z., et al. (2023). H2O: Heavy-hitter oracle for efficient generative inference of large language models. NeurIPS.